A4 Unity Catalog

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track A: Databricks pipeline engineering 2
A4 Unity Catalog 3
A4-L1 The three-level namespace, one catalog per environment 3
A4-L2 Grant to groups, grant the least, grant at the right level 7
A4-L3 Row filters and column masks in Unity Catalog 11
Track A best-practice checklist 16
Appendix: solutions and answer keys 17

Track A: Databricks pipeline engineering

Build Databricks pipelines that are correct, fast, observable and tested.

A4 Unity Catalog

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Three-level namespace, grants, row filters and column masks.

A4-L1 The three-level namespace, one catalog per environment

Concept

Unity Catalog names every table with three parts: `catalog.schema.table`.

```
1 research_prod.research_silver.nat_respondents
```

Use each level for one purpose:

- **Catalog = environment** (or a major business domain): `research_dev`, `research_test`, `research_prod`. Permissions and storage attach at this level, so a dev job with no rights on `research_prod` cannot damage it.
- **Schema = layer or subject**: `research_bronze`, `research_silver`, `research_gold`, `research_governance`.
- **Table = content at a stated grain**: `nat_raw`, `nat_respondents`, `nat_metrics`.

Always write the full three-part name in pipeline code, built from configuration. A two-part name such as `research_silver.nat_respondents` resolves against the session's **current catalog**. That depends on the workspace default, a `USE CATALOG` somewhere earlier in the notebook, or a job setting. The same code then reads a different table in each place it runs.

Build names in one function from config, so no code concatenates strings on its own:

```
1 def table_name(catalog: str, layer: str, tracker: str) -> str:
2     suffix = {"bronze": "raw", "silver": "respondents", "gold": "metrics"}
    [layer]
3     return f"{catalog}.research_{layer}.{tracker}_{suffix}"
```

In a notebook, `spark.catalog.currentCatalog()` tells you which catalog a two-part name would use. It is useful for checks, never as a substitute for naming the catalog.

Workspaces created before Unity Catalog also show `hive_metastore`, the legacy two-level store. Treat it as read-only history while you migrate.

Why it matters

A shared all-purpose cluster has `research_prod` as its default catalog. A developer tests a silver fix from a notebook that writes to `research_silver.nat_respondents`, a two-part name, believing the notebook "runs in dev". The MERGE lands in production. The fix was only half finished, and 1,400 rows get wrong region codes before anyone notices.

With a three-part name built from a dev config file, the same notebook writes to `research_dev.research_silver.nat_respondents`. The developer's lack of MODIFY on `research_prod` would also have blocked it.

Worked example: weak code

```
1  spark.sql("USE CATALOG research_prod")
2
3  def load_silver(spark):
2      return spark.table("research_silver.nat_respondents")
5
6  def write_gold(df):
3      df.write.mode("append").saveAsTable("research_gold." + "nat" +
       "_metrics")
```

1. **Line 1.** Changes the current catalog for everything after it in the session.
2. **Line 4.** Two-part name. Which catalog it reads depends on the session, not the code.
3. **Line 7.** Names glued together in several places. One typo and a new table appears.

Worked example: strong code

```
1  LAYER_SUFFIX = {"bronze": "raw", "silver": "respondents", "gold":
     "metrics"}
2
3
4  def table_name(catalog: str, layer: str, tracker: str) -> str:
5      """Three-level table name for a tracker and layer.
6
7      Raises:
8          KeyError: If layer is not bronze, silver or gold.
9      """
2      return f"{catalog}.research_{layer}.{tracker}_{LAYER_SUFFIX[layer]}"
11
12
13 def load_silver(spark: SparkSession, cfg: RunConfig) -> DataFrame:
14     """Read the silver table for the configured environment and
       tracker."""
3      return spark.table(table_name(cfg.catalog, "silver", cfg.tracker))
```

1. **Line 1.** The naming convention lives in one constant.

2. **Line 10.** Always three parts. An unknown layer raises KeyError instead of inventing a table.
3. **Line 15.** The environment comes from config. The same code reads dev in dev and prod in prod.

The strong version reads exactly the table its config names, whatever the session's current catalog is.

Recap card

- **Unity Catalog name format** catalog.schema.table, for example research_prod.research_silver.nat_respondents.
- **What does each level hold in this project?** Catalog: environment. Schema: layer or governance. Table: tracker at a grain.
- **Why never write a two-part table name in pipeline code?** It resolves against the session's current catalog, which varies by cluster, notebook and job.
- **Where do table names come from?** One function built from config, such as table_name(catalog, layer, tracker).

Practice

Exercise 1. Multiple choice (level 1)

Which level of the Unity Catalog namespace should separate dev from prod?

- a) Catalog
- b) Schema
- c) Table name prefix, such as dev_nat_metrics
- d) A column called environment

Why?

- r1) Permissions and storage attach at catalog level, so dev principals can be kept out of prod entirely.
- r2) Catalog names are shorter.

Exercise 2. Fill the gap (level 2)

Complete the fully qualified name of the gold table for tracker `nat` in production.

```
1 GOLD = "_____(1)._____(2).nat_metrics"
```

Exercise 3. Spot the bug (level 3)

This job runs in the dev workspace, but its writes appear in production. Tap the line, then fix the function.

```
1 def write_metrics(df: DataFrame, cfg: RunConfig) -> None:
2     """Write gold metrics for the configured environment."""
3     target = f"research_gold.{cfg.tracker}_metrics"
4     df.write.format("delta").mode("append").saveAsTable(target)
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 4)

Write `table_name(catalog, layer, tracker)`. It returns `catalog.research_<layer>.<tracker>_<suffix>` where the suffix is `raw`, `respondents` or `metrics` for bronze, silver and gold. Raise `ValueError` naming the layer for any other layer, and raise `ValueError` if `catalog` contains a dot.

Your code must pass these checks:

- Build catalog.research_<layer>.<tracker>_<suffix> with raw, respondents and metrics.
- An unknown layer must raise ValueError that names it.
- A catalog containing a dot would produce a four-part name. Raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
1 # Module constant.
2 LAYER_SUFFIX = ...
3
4 # Three-level Unity Catalog name for a tracker's table in one layer.
5 def table_name(
6     catalog: str,
7     layer: str,
8     tracker: str,
9 ) -> str: ...
```

Starter code:

```
1 def table_name(catalog: str, layer: str, tracker: str) -> str:
2     ...
```

Exercise 5. Architecture design (level 5)

You lay out Unity Catalog for the research programme. Choose the level for each item.

Options: Catalog, Schema, Table.

1. Development versus production

2. Bronze, silver and gold
3. Run log, variable registry and contracts
4. One tracker's respondents at respondent x wave grain

Sources

- Unity Catalog uses a three-level namespace, catalog.schema.table, and two-part names resolve against the current catalog. (verified: https://docs.databricks.com/aws/en/data-governance/unity-catalog/ (checked 2026-09-25))
- spark.catalog.currentCatalog() returns the current catalog (PySpark 3.4 and later). (verified: Present on PySpark 4.0.4 in this repository.)

A4-L2 Grant to groups, grant the least, grant at the right level

Concept

Unity Catalog controls access with privileges on securable objects. Four privileges cover most pipeline work:

Privilege	On	Lets the principal
`USE CATALOG`	catalog	see into the catalog. Grants no data access by itself.
`USE SCHEMA`	schema	see into the schema. Grants no data access by itself.
`SELECT`	table, or schema	read data
`MODIFY`	table, or schema	insert, update and delete data

To read a table, a principal needs all three: `USE CATALOG` on its catalog, `USE SCHEMA` on its schema and `SELECT` on the table. `MODIFY` also needs `SELECT` and both `USE` privileges.

Grant at the container. A privilege granted on a schema is inherited by every current and future table in it. `GRANT SELECT ON SCHEMA research_prod.research_gold` covers next month's new gold table without another ticket.

Grant to groups, never to individual users. People join, move and leave. A group grant is managed once, in one place. A user grant outlives the reason it was made, and nobody finds it until an audit.

Run pipelines as a service principal. Jobs that write bronze, silver and gold run as a service principal with `MODIFY` on those schemas. People get `SELECT` on the layers they need, and no person needs `MODIFY` on production.

```
1 GRANT USE CATALOG ON CATALOG research_prod TO `research_analysts`;
2 GRANT USE SCHEMA, SELECT ON SCHEMA research_prod.research_gold TO
    `research_analysts`;
3 SHOW GRANTS ON SCHEMA research_prod.research_gold;
```

Avoid `ALL PRIVILEGES` outside sandboxes. It includes privileges you did not think about, and future privileges too.

Why it matters

During a deadline, an analyst gets `MODIFY` on `research_prod.research_silver` "for one day" to patch a mapping by hand. The grant is to their user account. Eight months later they run a clean-up notebook against what they believe is a dev copy and delete 12,000 silver rows. Nobody remembered the grant, and silver had to be rebuilt from bronze.

With the rule "people get SELECT, pipelines get MODIFY, grants go to groups", the patch would have gone through the pipeline, and the delete would have failed with a permission error.

Worked example: weak code

```
1 GRANT ALL PRIVILEGES ON CATALOG research_prod TO `alex@example.org`;
2 GRANT SELECT ON TABLE research_prod.research_gold.nat_metrics TO
    `research_analysts`;
3 GRANT MODIFY ON SCHEMA research_prod.research_silver TO
    `research_analysts`;
```

1. **Line 1.** Everything, on all of production, to one person. Outlives the reason for it.
2. **Line 2.** SELECT alone is not enough, and a table-level grant misses every new gold table.
3. **Line 3.** Analysts can now change silver directly, bypassing the pipeline and its checks.

Worked example: strong code

```
1 -- Analysts read gold.
2 GRANT USE CATALOG ON CATALOG research_prod TO `research_analysts`;
1 GRANT USE SCHEMA, SELECT ON SCHEMA research_prod.research_gold TO
    `research_analysts`;
```

```
4
5  -- The pipeline service principal writes silver and gold.
6  GRANT USE CATALOG ON CATALOG research_prod TO `research-pipeline-sp`;
2  GRANT USE SCHEMA, SELECT, MODIFY ON SCHEMA research_prod.research_silver
     TO `research-pipeline-sp`;
8  GRANT USE SCHEMA, SELECT, MODIFY ON SCHEMA research_prod.research_gold TO
     `research-pipeline-sp`;
9
10 -- Check what is in place.
3  SHOW GRANTS ON SCHEMA research_prod.research_gold;
```

1. **Line 3.** Schema level: inherited by every current and future gold table.
2. **Line 7.** Only the pipeline identity writes to silver.
3. **Line 11.** Review grants from SQL, and keep these statements in version control.

Three rules: groups not users, schemas not tables, SELECT for people and MODIFY for pipelines.

Recap card

- **Privileges needed to read a table** USE CATALOG on the catalog, USE SCHEMA on the schema, SELECT on the table or schema.
- **Why grant SELECT on the schema rather than the table?** Schema grants are inherited by every current and future table in it.
- **Who gets MODIFY on production silver and gold?** The pipeline service principal. People get SELECT.
- **Why never grant to a user account?** User grants outlive the reason for them. Group membership is managed in one place.

Practice

Exercise 1. Put in order (level 1)

Analysts must read every table in `research_prod.research_gold`. Put the grants in order from the outermost object inwards.

- GRANT USE SCHEMA ON SCHEMA research_prod.research_gold TO `research_analysts`
- GRANT SELECT ON SCHEMA research_prod.research_gold TO `research_analysts`
- GRANT USE CATALOG ON CATALOG research_prod TO `research_analysts`

Exercise 2. Multiple choice (level 2)

An analyst has `SELECT` on `research_prod.research_gold.nat_metrics` and nothing else. What happens when they query the table?

- a) The query succeeds.
- b) The query fails, because they lack USE CATALOG and USE SCHEMA.
- c) The query returns an empty result.
- d) The query succeeds for columns without a mask.

Why?

- r1) Reading needs USE on the parent catalog and schema as well as SELECT on the table.
- r2) SELECT only works when granted on a schema.

Exercise 3. Spot the bug (level 3)

A reviewer rejects this grant script. Tap the line that breaks the "groups, not users" rule, then rewrite the script.

```
1 GRANT USE CATALOG ON CATALOG research_prod TO `research_analysts`;
2 GRANT USE SCHEMA, SELECT ON SCHEMA research_prod.research_gold TO
    `research_analysts`;
3 GRANT USE SCHEMA, SELECT ON SCHEMA research_prod.research_silver TO
    `sam.jones@example.org`;
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 4)

Write the grants for the pipeline service principal `research-pipeline-sp`: it must read and write every table in `research_prod.research_silver`, now and in future. Give it only what it needs, on the right objects.

Shape of the answer. The reference answer follows this outline. Replace each ... with your own code.

```
1 GRANT USE CATALOG ...
2 GRANT USE ...
```

Starter code:

```
1 -- Grants for research-pipeline-sp
```

Exercise 5. Architecture design (level 5)

Assign each principal the least privilege that lets them do their job.

Options: USE CATALOG, USE SCHEMA, SELECT on the schema, USE CATALOG, USE SCHEMA, SELECT, MODIFY on the schema, ALL PRIVILEGES on the catalog, Nothing on this schema.

1. research_analysts build Power BI reports from gold
2. research-pipeline-sp writes silver
3. research_analysts and the bronze schema
4. Platform administrators managing the metastore

Sources

- Reading a table needs USE CATALOG, USE SCHEMA and SELECT; MODIFY needs SELECT and both USE privileges; privileges granted on a schema are inherited by current and future tables. (search-only: https://docs.databricks.com/aws/en/data-governance/unity-catalog/manage-privileges/)
- GRANT accepts a comma-separated list of privileges, and SHOW GRANTS lists grants on a securable. (verified: https://docs.databricks.com/aws/en/sql/language-manual/security-grant and https://docs.databricks.com/aws/en/sql/language-manual/security-show-grant (checked 2026-09-25))

A4-L3 Row filters and column masks in Unity Catalog

Concept

Grants decide who can read a table. **Row filters** and **column masks** decide what each reader sees inside it. Both are SQL user-defined functions that Unity Catalog applies at query time, for every query, from every tool.

Row filter. A SQL UDF that returns BOOLEAN. Unity Catalog passes it the columns you name, and drops each row where the function returns FALSE or NULL.

```
1 CREATE FUNCTION
    research_prod.research_governance.respondent_filter(tracker_id STRING)
2 RETURN is_account_group_member('research_approved_analysts') OR
    tracker_id = 'TRK-PUBLIC';
3
4 ALTER TABLE research_prod.research_silver.nat_respondents
5 SET ROW FILTER research_prod.research_governance.respondent_filter ON
    (tracker_id);
```

Column mask. A SQL UDF whose first parameter is the column value. It returns the value, or a masked version.

```
CREATE FUNCTION research_prod.research_governance.mask_postcode(postcode
  STRING)
RETURN CASE WHEN is_account_group_member('pii_readers') THEN postcode
  ELSE '***' END;

ALTER TABLE research_prod.research_silver.nat_respondents
ALTER COLUMN postcode SET MASK
  research_prod.research_governance.mask_postcode;
```

Three rules:

- Use `is_account_group_member()`. It checks account-level groups, which Unity Catalog uses. `is_member()` checks workspace-local groups.
- Test group membership, never `current_user() = 'someone@example.org'`. A named user in a function is a hidden grant nobody reviews.
- Keep filter and mask functions in a governance schema, so their owners and grants are separate from the data.

Your reference script uses `CREATE ROW ACCESS POLICY`. That is Snowflake syntax. Databricks rejects it. Where many tables share one rule, Databricks also offers attribute-based policies on governed tags.

Why it matters

The reference `unity_catalog.sql` was written with an AI assistant and never run. Its row-level security uses `CREATE ROW ACCESS POLICY`, which does not exist in Databricks. The deployment script stops at that statement, and the PII table goes live without a filter, because the grants earlier in the script had already succeeded. Every analyst with SELECT on silver can read restricted respondent rows until someone notices the failed step.

A SQL UDF with `SET ROW FILTER`, tested in dev, applies the rule where Databricks enforces it.

Worked example: weak code

```
1 CREATE ROW ACCESS POLICY restrict_respondent_data
2     ON research_silver.nat_respondents
3     USING (
4         is_member('research_approved_analysts')
5         OR current_user() = 'pipeline-svc@example.org'
6     );
```

1. **Line 1.** Snowflake syntax. Databricks raises a parse error. Reference defect 4.
2. **Line 2.** Two-part name: resolves against whatever catalog is current.
3. **Line 4.** Checks workspace-local groups. Unity Catalog groups are account-level.
4. **Line 5.** A named identity inside a function: a grant hidden from every access review.

Worked example: strong code

```
1  CREATE OR REPLACE FUNCTION
     research_prod.research_governance.respondent_filter(tracker_id STRING)
2  RETURN is_account_group_member('research_approved_analysts')
2    OR is_account_group_member('research_pipelines');
4
5  ALTER TABLE research_prod.research_silver.nat_respondents
3  SET ROW FILTER research_prod.research_governance.respondent_filter ON
     (tracker_id);
7
8  CREATE OR REPLACE FUNCTION
     research_prod.research_governance.mask_postcode(postcode STRING)
9  RETURN CASE WHEN is_account_group_member('pii_readers') THEN postcode
     ELSE '***' END;
10
11 ALTER TABLE research_prod.research_silver.nat_respondents
4  ALTER COLUMN postcode SET MASK
     research_prod.research_governance.mask_postcode;
```

1. **Line 1.** A SQL UDF in the governance schema. Its parameter receives the column named in ON (...).
2. **Line 3.** The pipeline service principal is in a group. No identity is named in the function.
3. **Line 6.** Binds the filter to the table. It applies to every query from every tool.
4. **Line 12.** Masks one column. Readers outside pii_readers see ***.

The strong script runs on Databricks, names groups instead of people, and uses three-part names throughout.

Recap card

- **Row filter in Databricks** A SQL UDF returning BOOLEAN, bound with ALTER TABLE t SET ROW FILTER f ON (col).
- **Column mask in Databricks** A SQL UDF taking the column value, bound with ALTER TABLE t ALTER COLUMN c SET MASK f.
- **is_account_group_member or is_member?** is_account_group_member: Unity Catalog groups are account-level. is_member checks workspace groups.
- **What is CREATE ROW ACCESS POLICY?** Snowflake syntax. Databricks rejects it. Reference defect 4.

Practice

Exercise 1. Multiple choice (level 1)

Which statement applies a row filter to a Unity Catalog table?

- a) `CREATE ROW ACCESS POLICY p ON t USING (...)`
- b) `ALTER TABLE t SET ROW FILTER f ON (col)`
- c) `GRANT SELECT ON t TO g WHERE col = 'x'`
- d) `CREATE VIEW v AS SELECT * FROM t WHERE ...` and nothing else

Why?

- r1) Databricks row filters are SQL UDFs bound to a table with SET ROW FILTER.
- r2) Row filters must be created before the table.

Exercise 2. Fill the gap (level 2)

Complete the mask so that only members of the account group `pii_readers` see real postcodes.

```
CREATE FUNCTION research_prod.research_governance.mask_postcode(postcode
  STRING)
RETURN CASE WHEN _____(1)('pii_readers') THEN postcode ELSE '***' END;

ALTER TABLE research_prod.research_silver.nat_respondents
ALTER COLUMN postcode _____(2)
  research_prod.research_governance.mask_postcode;
```

Exercise 3. Spot the bug (level 3)

An access review flags this filter. Tap the line that hides a grant from reviewers, then rewrite the function.

```
CREATE OR REPLACE FUNCTION
  research_prod.research_governance.respondent_filter(tracker_id STRING)
RETURN is_account_group_member('research_approved_analysts')
   OR current_user() = 'pipeline-svc@example.org';
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 4)

Write a row filter for `research_prod.research_gold.nat_metrics`. Members of `research_all_regions` see every row. Everyone else sees only rows where `region = 'NATIONAL'`. Create the function `research_prod.research_governance.region_filter` and bind it to the table on the `region` column.

Shape of the answer. The reference answer follows this outline. Replace each ... with your own code.

```
CREATE OR REPLACE ...
RETURN ...

ALTER TABLE ...
SET ROW FILTER ...
```

Starter code:

```
CREATE OR REPLACE FUNCTION
  research_prod.research_governance.region_filter(region STRING)
RETURN ...
```

Exercise 5. Architecture design (level 5)

Choose the control for each requirement on `nat_respondents`.

Options: Grants (or their absence), Row filter, Column mask.

1. The client team must not query the table at all
2. Analysts see only the trackers their group is approved for
3. Analysts see respondent rows, but postcodes appear as *** unless they are in pii_readers

Sources

- Row filters are SQL UDFs returning BOOLEAN bound with SET ROW FILTER ... ON (cols); column masks are bound with ALTER COLUMN ... SET MASK. (verified: https://docs.databricks.com/aws/en/data-governance/unity-catalog/filters-and-masks/manually-apply (checked 2026-09-25))
- is_account_group_member returns true when the session user is a direct or indirect member of an account-level group; is_member checks workspace groups. (verified: https://docs.databricks.com/aws/en/sql/language-manual/functions/is_account_group_member (checked 2026-09-25))
- Attribute-based access control (ABAC) policies apply filters and masks centrally through governed tags. (verified: https://docs.databricks.com/aws/en/data-governance/unity-catalog/abac/policies (checked 2026-09-25))

Track A best-practice checklist

☐ Every function has type hints, a docstring and one job.

☐ No SparkSession is created at import time. Functions receive spark as a parameter.

☐ Functions are imported as F: `from pyspark.sql import functions as F`.

☐ Table names, column maps and thresholds come from config or registry tables.

☐ Every table states its grain in a comment or table property.

☐ Bronze appends. Silver deduplicates the source, then MERGEs. Gold aggregates by every grain column.

☐ Every run writes one record to the run log table with an explicit schema.

☐ CRITICAL checks raise. WARN checks log and continue.

☐ Every division has a guard for zero and null.

☐ Counts come from one agg pass, not repeated count() calls.

☐ Incremental loads compare source and target on the same keys.

☐ Row filters use a SQL UDF with SET ROW FILTER. Grants go to groups.

☐ Tests run on a local SparkSession in CI before every merge.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

A4-L1 The three-level namespace, one catalog per environment

A4-L1 exercise 1

Hints. (1) Which level can you lock a whole environment behind? (2) Grants on the top level control everything below it. (3) The catalog.

Answer. a) Catalog Because: r1) Permissions and storage attach at catalog level, so dev principals can be kept out of prod entirely.

One catalog per environment gives a hard boundary. A dev job without privileges on `research_prod` cannot read or write it by accident.

Common wrong answers.

- `b`: Schemas are better used for layers. Putting environments there mixes dev and prod under one catalog's permissions.
- `c`: A prefix is a naming habit, not a boundary. Anyone with rights on the schema can write both.
- `d`: A column puts dev and prod rows in the same table. One bad filter mixes them.
- `a|r2`: Right level, wrong reason. The point is isolation: grants on research_prod do not reach research_dev, and the reverse.

A4-L1 exercise 2

Hints. (1) catalog.schema.table. (2) The environment is the catalog. (3) research_prod, then research_gold.

Answer.

```
GOLD = "research_prod.research_gold.nat_metrics"
```

`research_prod.research_gold.nat_metrics`: environment, layer, table.

Common wrong answers.

- g1 = `research_gold`; g2 = `research_prod`: Swapped. The catalog comes first and holds the environment. The schema holds the layer.
- g1 = `hive_metastore`: hive_metastore is the legacy store. Production tables live in a Unity Catalog catalog such as research_prod.

A4-L1 exercise 3

Hints. (1) Count the parts in the table name. (2) What decides the catalog for a two-part name? (3) Add `cfg.catalog` as the first part.

Answer. The bug is on line 3.

Fixed version:

```
1 def write_metrics(df: DataFrame, cfg: RunConfig) -> None:
2     """Write gold metrics for the configured environment."""
3     target = f"{cfg.catalog}.research_gold.{cfg.tracker}_metrics"
4     df.write.format("delta").mode("append").saveAsTable(target)
```

A two-part name resolves against the session's current catalog, which here is production. Build a three-part name from `cfg.catalog`.

Common wrong answers.

- Line 4: Append is right for this job. The question is which table it writes to.

A4-L1 exercise 4

Hints. (1) Keep the suffixes in a dict constant. (2) Check for a dot in catalog and for an unknown layer before building the name. (3) `f"{catalog}.research_{layer}.{tracker}_{LAYER_SUFFIX[layer]}"`.

Answer.

```
1  LAYER_SUFFIX = {"bronze": "raw", "silver": "respondents", "gold":
       "metrics"}
2
3
4  def table_name(catalog: str, layer: str, tracker: str) -> str:
5      """Three-level Unity Catalog name for a tracker's table in one layer.
6
7      Args:
8          catalog: Environment catalog, for example research_dev.
9          layer: bronze, silver or gold.
10         tracker: Tracker short name, for example nat.
11
12     Returns:
13         The fully qualified table name.
14
15     Raises:
16         ValueError: If the layer is unknown or the catalog contains a dot.
17     """
18     if "." in catalog:
19         raise ValueError(f"catalog must be a single name, got {catalog!
       r}")
20     if layer not in LAYER_SUFFIX:
21         raise ValueError(f"unknown layer {layer!r}")
22     return f"{catalog}.research_{layer}.{tracker}_{LAYER_SUFFIX[layer]}"
```

One function owns the naming convention. It rejects inputs that would produce a wrong name instead of silently creating a new table.

A4-L1 exercise 5

Hints. (1) The strongest boundary goes at the top. (2) Layers group tables with the same contract. (3) Tables hold one dataset at one grain.

Answer.

- Catalog per environment: `research_dev`, `research_prod`.
- Schema per layer, plus `research_governance` for the run log, registry and contracts.
- Table per tracker and grain: `nat_respondents`.

Environment, layer, table: each level answers one question, and the full name tells a reader all three.

Common wrong answers.

- layer = `catalog`: A catalog per layer multiplies catalogs by environments, and grants must then be repeated everywhere. Use schemas for layers.
- tracker = `schema`: A schema per tracker scatters the layers. Keep layers as schemas, and trackers as tables within them.
- env = `schema`: Schemas under one catalog share its permissions and storage. Environments need the stronger boundary of separate catalogs.

A4-L2 Grant to groups, grant the least, grant at the right level

A4-L2 exercise 1

Hints. (1) The namespace goes catalog, schema, table. (2) Each USE privilege opens one level. (3) The data privilege comes last.

Answer.

1. GRANT USE CATALOG ON CATALOG research_prod TO `research_analysts`
2. GRANT USE SCHEMA ON SCHEMA research_prod.research_gold TO `research_analysts`
3. GRANT SELECT ON SCHEMA research_prod.research_gold TO `research_analysts`

Reading a table needs USE CATALOG, USE SCHEMA and SELECT. Granting SELECT on the schema makes every current and future table readable.

Common wrong answers.

- The grants all work in any order, but read them outermost first: catalog, then schema, then data. This order is inside out.
- Catalog comes before schema. USE CATALOG opens the catalog, USE SCHEMA opens the schema inside it.

A4-L2 exercise 2

Hints. (1) How many privileges does a read need? (2) Each parent level must be usable. (3) USE CATALOG and USE SCHEMA are missing.

Answer. b) The query fails, because they lack USE CATALOG and USE SCHEMA. Because: r1) Reading needs USE on the parent catalog and schema as well as SELECT on the table.

A read needs USE CATALOG, USE SCHEMA and SELECT. With SELECT alone the query fails with a permission error.

Common wrong answers.

- `a`: SELECT on the table is necessary but not enough. The catalog and schema must be usable too.
- `c`: Missing privileges raise a permission error. They never return an empty result.
- `d`: Masks are unrelated. The query fails before any column is read.
- `b|r2`: Right outcome, wrong reason. SELECT can be granted on a table. The missing pieces are USE CATALOG and USE SCHEMA.

A4-L2 exercise 3

Hints. (1) Which principal is not a group? (2) An email address names one person. (3) Create a group such as research_silver_readers and grant to it.

Answer. The bug is on line 3.

Fixed version:

```
1 GRANT USE CATALOG ON CATALOG research_prod TO `research_analysts`;
2 GRANT USE SCHEMA, SELECT ON SCHEMA research_prod.research_gold TO
    `research_analysts`;
3 GRANT USE CATALOG ON CATALOG research_prod TO `research_silver_readers`;
4 GRANT USE SCHEMA, SELECT ON SCHEMA research_prod.research_silver TO
    `research_silver_readers`;
```

Line 3 grants to one person. Grant to a group instead, and add Sam to the group. When Sam moves teams, one membership change removes the access.

Common wrong answers.

- Line 2: That grant goes to a group at schema level, which is the pattern to follow.
- Line 1: USE CATALOG to a group is correct and needed.

A4-L2 exercise 4

Hints. (1) Two statements are enough. (2) One for the catalog, one for the schema. (3) `GRANT USE SCHEMA, SELECT, MODIFY ON SCHEMA research_prod.research_silver TO ...`.

Answer.

```
GRANT USE CATALOG ON CATALOG research_prod TO `research-pipeline-sp`;
GRANT USE SCHEMA, SELECT, MODIFY ON SCHEMA research_prod.research_silver
    TO `research-pipeline-sp`;
```

USE CATALOG opens the catalog. USE SCHEMA, SELECT and MODIFY on the schema cover every table in it, now and in future. Nothing else is needed.

A4-L2 exercise 5

Hints. (1) Who writes, and who only reads? (2) Does anyone in the list need raw deliveries? (3) Groups, not users, for every grant.

Answer.

- Analysts read gold: USE CATALOG, USE SCHEMA and SELECT on the gold schema.
- The pipeline writes silver: the same plus MODIFY on the silver schema.
- Analysts have no reason to see raw deliveries, so no bronze access.
- Administration goes through an admin group, never through grants on user accounts.

Least privilege: people read the layer they report from, pipelines write the layers they own, and admin rights live in a group.

Common wrong answers.

- analysts = `modify`: Reports only read. MODIFY would let an analyst change gold outside the pipeline.
- analysts_bronze = `select`: Bronze holds raw deliveries, which can include fields gold never exposes. Give access only when there is a need.
- admin = `user_all`: Grants to user accounts outlive the people's roles. Use a group.

A4-L3 Row filters and column masks in Unity Catalog

A4-L3 exercise 1

Hints. (1) The filter is a SQL function. (2) You attach it with ALTER TABLE. (3) SET ROW FILTER ... ON (column).

Answer. b) `ALTER TABLE t SET ROW FILTER f ON (col)` Because: r1) Databricks row filters are SQL UDFs bound to a table with SET ROW FILTER.

Create a SQL UDF that returns BOOLEAN, then bind it with `ALTER TABLE ... SET ROW FILTER f ON (col)`.

Common wrong answers.

- `a`: That is Snowflake syntax. Databricks raises a parse error on it.
- `c`: GRANT has no WHERE clause. Grants control the whole object.
- `d`: A view filters only readers who use the view. Anyone with SELECT on the table bypasses it. A row filter applies to the table itself.
- `b|r2`: Right statement, wrong reason. You can add a row filter to an existing table with ALTER TABLE.

A4-L3 exercise 2

Hints. (1) The function checks account-level group membership. (2) The column clause has two words. (3) `is_account_group_member` and `SET MASK`.

Answer.

```
CREATE FUNCTION research_prod.research_governance.mask_postcode(postcode
    STRING)
RETURN CASE WHEN is_account_group_member('pii_readers') THEN postcode
    ELSE '***' END;

ALTER TABLE research_prod.research_silver.nat_respondents
ALTER COLUMN postcode SET MASK
    research_prod.research_governance.mask_postcode;
```

`is_account_group_member('pii_readers')` checks the account group. `ALTER COLUMN ... SET MASK f` binds the mask to the column.

Common wrong answers.

- g1 = `is_member`: `is_member` checks workspace-local groups. Unity Catalog uses account-level groups: `is_account_group_member`.
- g1 = `current_user`: `current_user()` returns a name, not a group check. Masks should test group membership.
- g2 = `SET ROW FILTER`: Row filters apply to the table. A column uses SET MASK.

A4-L3 exercise 3

Hints. (1) Which line names one identity? (2) Access reviews look at groups and grants, not at function bodies. (3) Replace the email check with a group check.

Answer. The bug is on line 3.

Fixed version:

```
1 CREATE OR REPLACE FUNCTION
    research_prod.research_governance.respondent_filter(tracker_id STRING)
2 RETURN is_account_group_member('research_approved_analysts')
3    OR is_account_group_member('research_pipelines');
```

Line 3 grants access to one named identity inside the function body, where no access review looks. Put the service principal in a group and test the group.

Common wrong answers.

- Line 2: Testing an account group is the right pattern. Look for a named person or identity.
- Line 1: A governance schema is the right home for the function.

A4-L3 exercise 4

Hints. (1) The function returns TRUE for rows the reader may see. (2) Group members see everything, so the group check stands alone on one side of OR. (3) Then bind with ALTER TABLE ... SET ROW FILTER ... ON (region).

Answer.

```
1 CREATE OR REPLACE FUNCTION
    research_prod.research_governance.region_filter(region STRING)
2 RETURN is_account_group_member('research_all_regions') OR region =
    'NATIONAL';
3
4 ALTER TABLE research_prod.research_gold.nat_metrics
5 SET ROW FILTER research_prod.research_governance.region_filter ON
    (region);
```

`group OR region = 'NATIONAL'` is TRUE for every row when the reader is in the group, and only for national rows otherwise. `AND` would hide everything from people outside the group. The function does nothing until `SET ROW FILTER` binds it.

A4-L3 exercise 5

Hints. (1) Whole table, some rows, one column. (2) Grants are all or nothing on an object. (3) Filters remove rows. Masks change values.

Answer.

- No access at all: do not grant SELECT. Grants decide who reads the table.
- Some rows only: a row filter decides which rows each reader sees.
- All rows, hidden values: a column mask decides what each reader sees in one column.

Grants control access to the object, filters control rows, masks control column values. Combine them: grant SELECT to analysts, filter trackers, mask postcodes.

Common wrong answers.

- team = `filter`: A filter that returns nothing still lets the team query the table and see its structure. Withhold the grant.
- tracker = `mask`: A mask changes values in a column. The requirement removes whole rows.
- postcode = `filter`: Filtering would remove the respondents entirely. The requirement keeps the rows and hides one column.